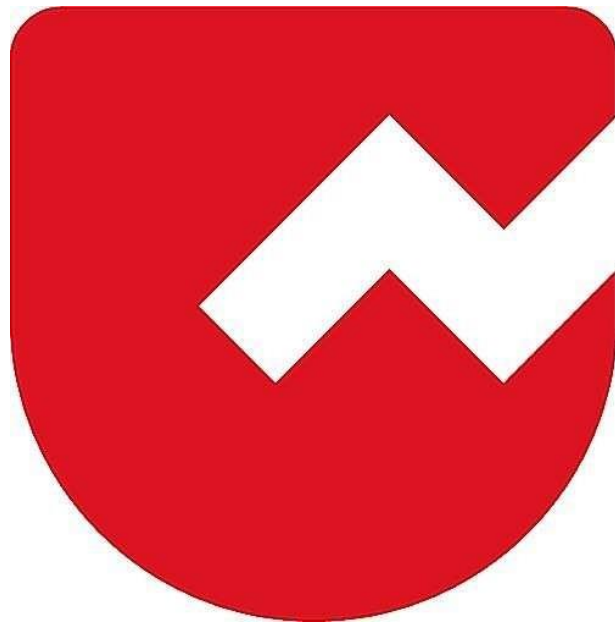




Zoho Schools for Graduate Studies



Notes

METHODS

1. What is Method ?

Enclosing set of statement that performs unique task.

2. Syntax of Methods

```
[access modifier] [non-access modifier] returnType <methodName>([parameterList]) [throws]
[Exception list] {
    // set of statements
}
```

3. Why we need a method or what is the primary purpose of a method?

- **Code Re-usability:** A method needs to be written only once, but it can be invoked multiple times from different places in the program whenever required.
- **Modularity:** Modularity breaking program into parts. Methods divide a large program into smaller, manageable, and organized pieces.

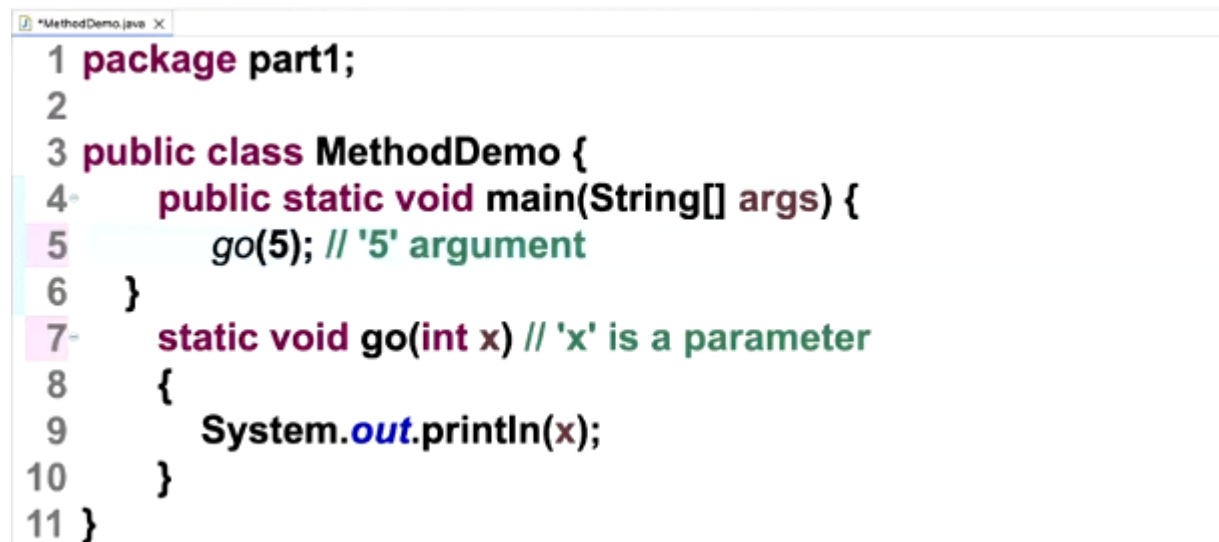
4. What is the advantages of Modularity?

- Easy debugging
- Improves code readability

5. Where generally in the program logic we write?

- All logic and executable statements are written inside methods.
- Business logic resides inside a method.

6. Parameter and Arguments

A screenshot of a Java code editor window titled '*MethodDemo.java'. The code is as follows:

```
1 package part1;
2
3 public class MethodDemo {
4     public static void main(String[] args) {
5         go(5); // '5' argument
6     }
7     static void go(int x) // 'x' is a parameter
8     {
9         System.out.println(x);
10    }
11 }
```

7. What is the difference between parameter and arguments

- Parameters are variable(that stores the copy of an arguments)
- Arguments are data or value.

8. How many parameters method can have ?

- N numbers of parameters

9. Variable Length Arguments

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         go(5); // '5' argument
6     }
7     static void go(int x) // 'x' is a parameter
8     {
9         System.out.println(x);
10    }
11 }

```

- varargs allow a method to accept zero or more arguments of the same type.
- Instead of fixing the number of parameters, you use ... (three dots) after the type.

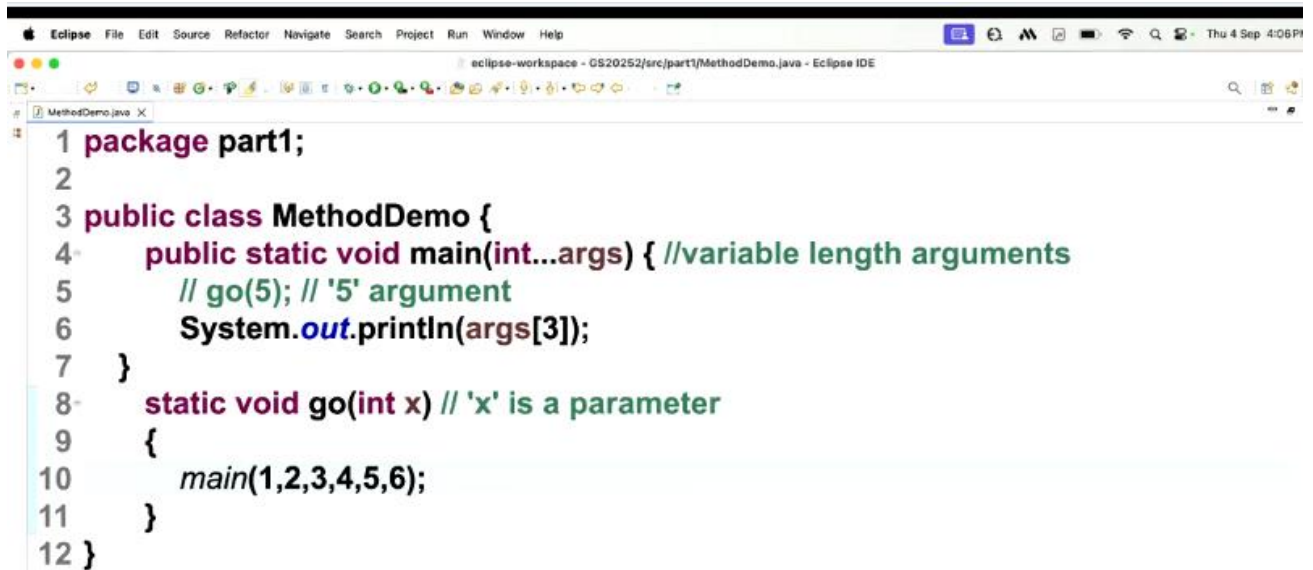
10. Can I call the main method from another method?

- Yes, It is perfectly legal to call the main method from another method. but the method which invoke the main method should be static.

```

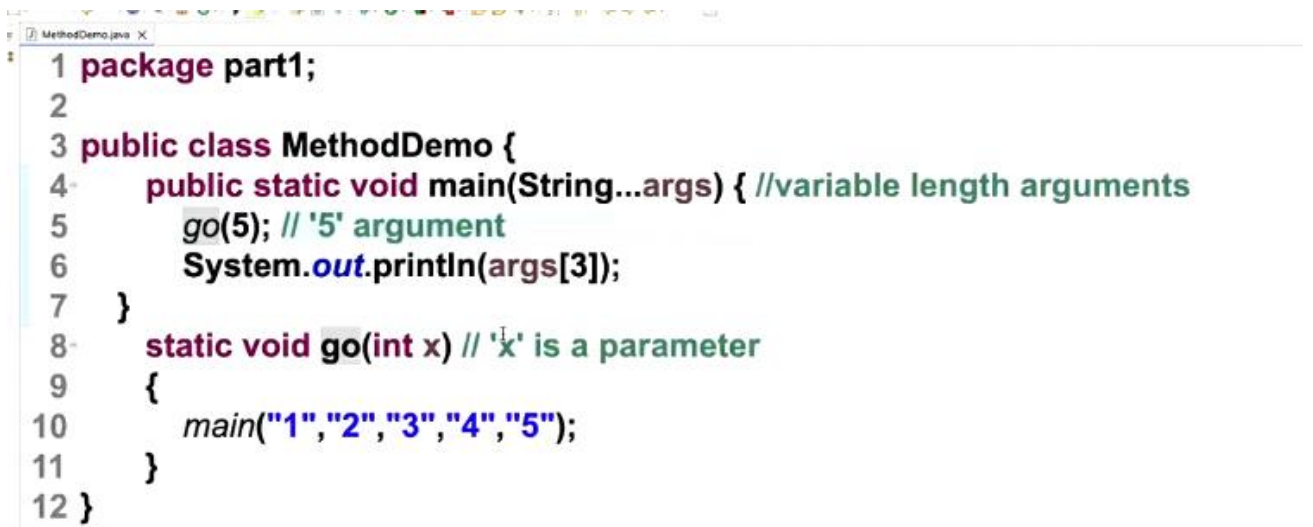
1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         // go(5); // '5' argument
6         System.out.println(args[3]);
7     }
8     static void go(int x) // 'x' is a parameter
9     {
10         main("1","2","3","4","5");
11     }
12 }

```



```
1 package part1;
2
3 public class MethodDemo {
4     public static void main(int...args) { //variable length arguments
5         // go(5); // '5' argument
6         System.out.println(args[3]);
7     }
8     static void go(int x) // 'x' is a parameter
9     {
10         main(1,2,3,4,5,6);
11     }
12 }
```

- **Output :** Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 3.
- Because, The statement `main("1", "2", "3", "4", "5");` is inside `go(int x)` but `go()` is never called.



```
1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         go(5); // '5' argument
6         System.out.println(args[3]);
7     }
8     static void go(int x) // 'x' is a parameter
9     {
10         main("1","2","3","4","5");
11     }
12 }
```

- **Output :** Exception in thread "main" java.lang.StackOverflowError
- Because there's no stopping condition, the program will keep calling `main` and `go` until the stack overflows.

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4,5);
6     }
7     static void ho(int...a)
8     {
9         System.out.println(a[3]);
10    }
11 }

```

Output : 4

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4,5);
6     }
7     static void ho(int...a)
8     {
9         System.out.println(a.length);
10    }
11 }

```

Output : 5

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4);
6     }
7     static void ho(int b, int...a)
8     {
9         System.out.println(a.length + b);
10    }
11 }

```

Output : 4

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4,5);
6     }
7     static void ho(int...a,int...b)
8     {
9         System.out.println(a.length + b);
10    }
11 }

```

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4,5);
6     }
7     static void ho(int...a,int b)
8     {
9         System.out.println(a.length + b);
10    }
11 }

```

These code will not compile.
Because,

RULE 1 : when you use variable-length arguments it must be the last parameter in the method signature.

RULE 2 : A method can have at most one varargs parameter in its parentheses.

11. Can I declare a method with default keyword?

Yes can, but the method only appear in the interface, In other words Interface alone can have default methods.

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4,5);
6     }
7     static String ho(int...a)
8     {
9         System.out.println(a.length);
10        return ;
11    }
12 }

```

Output : the code will not compile.
Because, Null is not a String.

If you want to return a empty string

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4,5);
6     }
7     static String ho(int...a)
8     {
9         System.out.println(a.length);
10        return "";
11    }
12 }

```

```

1 package part1;
2
3 public class MethodDemo {
4     public static void main(String...args) { //variable length arguments
5         ho(1,2,3,4,5);
6     }
7     static String ho(int...a)
8     {
9         System.out.println(a.length);
10        return null;
11    }
12 }

```

- It is perfectly legal for a method to return `null`.
- `NULL` -> Means Undefined or Unknown

12. Can I interchange access modifier and non-access modifier ?

- Non-access modifier can appear before Access modifier.